Part 7 — Solr Administration

I created the SolrCloud collection app_logs on the two Solr 9.8 pods, defined the four fields the task asks for, indexed 50,000 sample log documents, and ran a time-range query, a full-text query, and a facet on level. The Admin UI is http://10.1.0.213:8983/solr/ .

The collection

Solr is already running as solr-0 (10.244.2.34 on hadoop-lab-worker2) and solr-1 (10.244.1.28 on hadoop-lab-worker). Both use the existing ZooKeeper ensemble. I created the collection from solr-0 with two shards and two replicas. maxShardsPerNode=2 is required here: four replicas on two nodes, and the default limit is one replica per node.

```
kubectl exec -n hadoop solr-0 -- curl -s
"http://localhost:8983/solr/admin/collections?action=CREATE&name=app_logs&numShards=2&replicationFact
or=2&maxShardsPerNode=2&wt=json"
```

The create returned status 0 in 18.7 seconds and registered four cores. The DELETE that the setup script runs first returned 400, Could not find collection : app_logs, because this was the first create. Both shard leaders landed on solr-0. solr-1 holds the active follower of each shard. That is still two shards and two replicas. shard1 holds about 25.1k documents and shard2 about 24.9k.

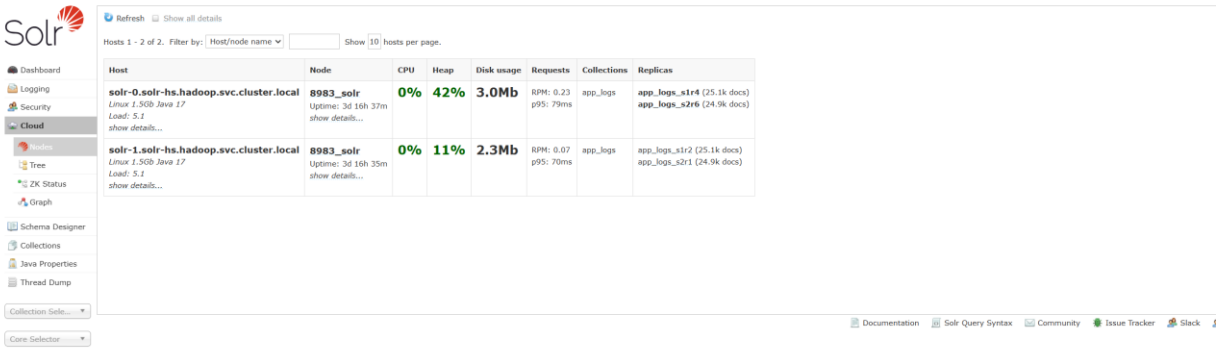


Figure 1. Cloud nodes. Each host has one replica of shard1 and one replica of shard2.

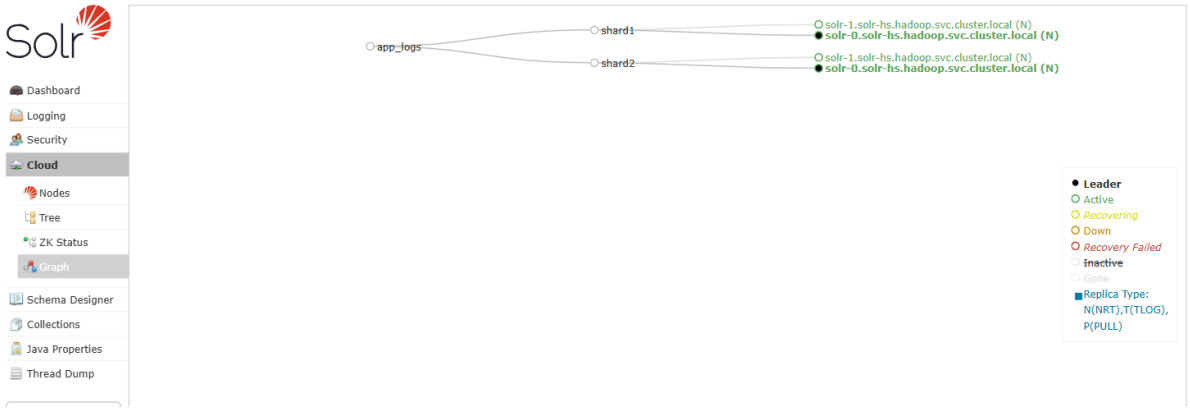


Figure 2. Cloud graph. Filled circles are the leaders on solr-0. Open circles are the active replicas on solr-1.

Schema

The schema file is scripts/part7-solr/app_logs_schema.json. The payload I posted is add_fields.json, which is the same list without id. id is already the unique key on the _default config, and adding it again makes the whole request fail. timestamp uses the built-in type pdate. That type's class is solr.DatePointField. level is a string with docValues, which is what the facet reads. message is text_general and single valued.

```
kubectl cp add_fields.json hadoop/solr-0:/tmp/add_fields.json
kubectl exec -n hadoop solr-0 -- curl -s -X POST -H "Content-type: application/json" --data-binary
@/tmp/add_fields.json "http://localhost:8983/solr/app_logs/schema"
```

The first add left message multi-valued, so stored values came back as a JSON array. I replaced the field with multiValued false, deleted every document, and indexed again. The Schema page below is after that replace.

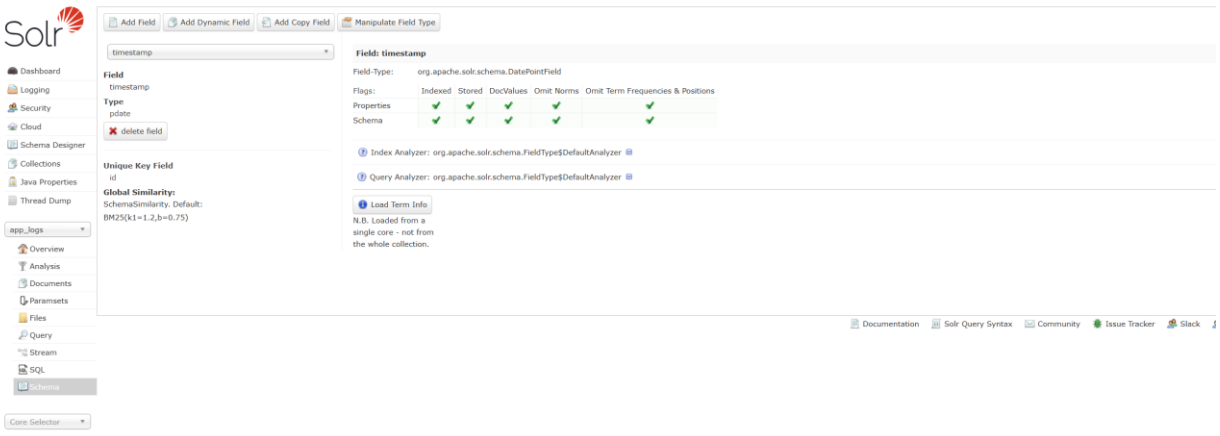


Figure 3. Field timestamp, type pdate, class org.apache.solr.schema.DatePointField.

Indexing 50,000 documents

The indexer is scripts/part7-solr/index_app_logs.py, also copied to /opt/hadoop-lab/solr/scripts/. It posts batches of 2,000 and commits at the end. Document ids run from log-000000 to log-049999. Every tenth document is ERROR and the next is WARN, so the collection has 5,000 ERROR, 5,000 WARN, and 40,000 INFO. Four of every ten timestamps fall inside the last seven days, measured from the NOW printed at the end of the load. Every fifth message contains the word timeout.

hadoop-env 05-index-result.png

```
$ python3 /opt/hadoop-lab/solr/scripts/index_app_logs.py \
    --url http://127.0.0.1:8983/solr/app_logs
sent 2000
sent 4000
...
sent 50000
RESULT indexed=50000 seconds=47.6 url=http://127.0.0.1:8983/solr/app_logs
NOW 2026-10-19T07:03:50Z
INDEX_EXIT:0
```

Figure 4. The final load. 50,000 documents in 47.6 seconds.

NOW for the queries below is 2026-10-19T07:03:50Z. A last-seven-days range is counted from that instant.

Time range on the last 7 days

I asked for timestamp from NOW-7DAYS to NOW and returned no stored rows, so the timing is the search itself.

```
curl -sG "$BASE/select" \
  --data-urlencode "q=timestamp:[NOW-7DAYS TO NOW]" \
  --data-urlencode "rows=0" --data-urlencode "wt=json"
```

hadoop-env 06-time-range.png

```
===== time-range-in-q =====
q = timestamp:[NOW-7DAYS TO NOW]    rows = 0
responseHeader.status = 0
responseHeader.QTime  = 27
response.numFound     = 20000
response.numFoundExact = true
```

Figure 5. Time-range query. 20,000 of the 50,000 documents fall in the window. QTime 27 ms on this capture.

Full-text search on message

text_general has already split each message into words, so I searched the token timeout. The Admin Query page shows the request, QTime, and two hits.

```
curl -sG "$BASE/select" \
  --data-urlencode "q=message:timeout" \
```

```
  --data-urlencode "rows=2" \
  --data-urlencode "fl=id,level,timestamp,message"
```

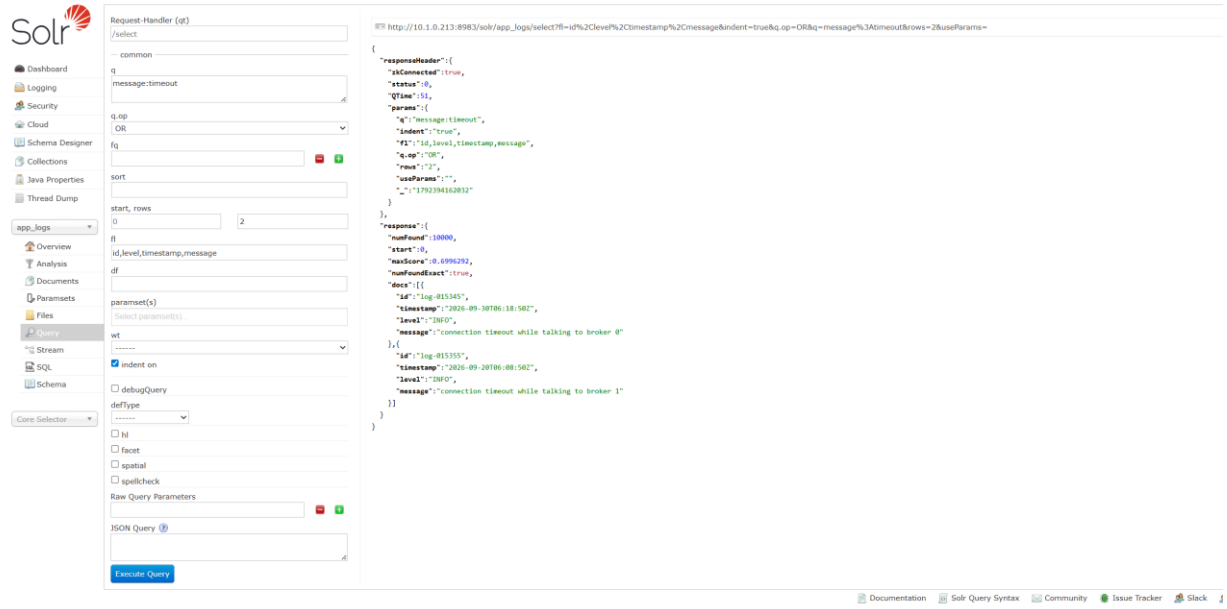


Figure 6. Admin Query. message:timeout matches 10,000 documents. QTime 55 ms on this click, with two sample rows.

Facet by level

The third query matches every document and groups the level field.

```
curl -sG "$BASE/select" \
  --data-urlencode "q=*:*" --data-urlencode "rows=0" \
  --data-urlencode "facet=true" --data-urlencode "facet.field=level"
```

hadoop-env 07-facet-level.png

```
===== facet-level =====
q = *:*    rows = 0    facet = true    facet.field = level
responseHeader.QTime = 56
response.numFound    = 50000
facet_fields.level:
  INFO   40000
  ERROR   5000
  WARN    5000
```

Figure 7. Facet on level. INFO 40,000, ERROR 5,000, WARN 5,000. QTime 56 ms on this capture.

One optimization for each query

QTime on 50,000 documents jumps around by tens of milliseconds, so I ran each pair three times with rows=0. The sample message query in Figure 6 uses rows=2, which also fetches stored fields, so I did not use that 55 ms number in the comparison.

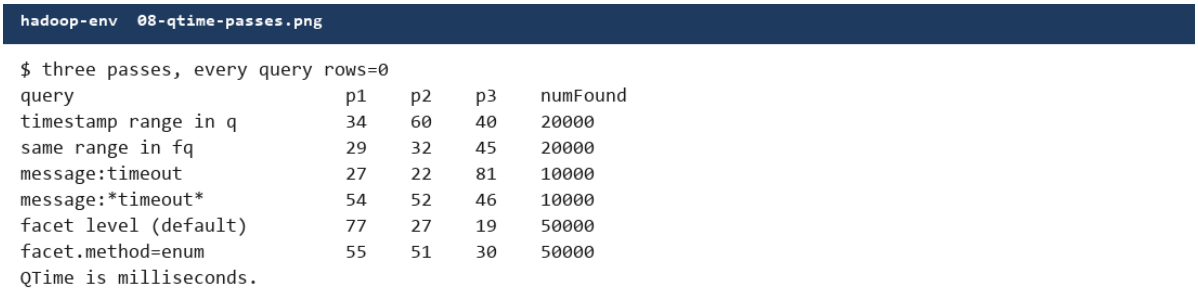


Figure 8. Three timing passes. QTime is milliseconds.

Query	Pass 1	Pass 2	Pass 3	Hits
Range in q	34 ms	60 ms	40 ms	20,000
Same range in fq	29 ms	32 ms	45 ms	20,000
message:timeout	27 ms	22 ms	81 ms	10,000
message:*timeout*	54 ms	52 ms	46 ms	10,000
Facet, default method	77 ms	27 ms	19 ms	50,000
facet.method=enum	55 ms	51 ms	30 ms	50,000

Time range. I would put the window in fq instead of q. A filter is cached and is not scored. Passes 1 and 2 were faster that way, 29 ms against 34 ms and 32 ms against 60 ms. Pass 3 went the other way, 45 ms against 40 ms. On this collection the gain is a few milliseconds, and it is not on every call.

Full text. I would keep the query as the token timeout. A leading star in message:*timeout* cannot use the word index, so Solr walks terms. Passes 1 and 2 were 27 ms and 22 ms for the token, against 54 ms and 52 ms for the wildcard. Pass 3 the token query spiked to 81 ms while the wildcard was 46 ms. Both forms return 10,000 documents. I still write the token query, and I do not treat that one slow pass as a reason to use the star.

Facet. facet.method=enum is the method for a field that has a few values when the collection is large. On these 50,000 documents it won only the first pass, 55 ms against 77 ms. The next two passes the default method was faster, 27 ms against 51 ms and 19 ms against 30 ms. Both methods returned INFO 40,000, ERROR 5,000, WARN 5,000. I would leave the default method at this size, and switch to enum when the index is much larger and level still has three values.

Where the files are

The schema, the indexer, and queries.sh are in scripts/part7-solr/ and on the VM at /opt/hadoop-lab/solr/scripts/. queries.sh prints all six calls: the three queries and the one change for each. The port-forward for the Admin UI is kubectl port-forward -n hadoop pod/solr-0 8983:8983. create_app_logs.sh deletes app_logs before it creates it, so I will not run that script again unless I intend to drop the 50,000 documents.